

Introduction to Bash

A Beginner's Guide for University Students

Software Engineering Tools - Automation

What is Bash?

- **Bourne Again SHell**
- Command-line interpreter and scripting language
- Default shell on most Unix/Linux systems and macOS
- Interface between user and operating system

Why Learn Bash?

- **Automation** of repetitive tasks
- **File management** and processing
- **System administration**
- **Development workflow** enhancement

Getting Started

Opening Terminal

- **macOS:** Applications → Utilities → Terminal
- **Linux:** Ctrl + Alt + T
- **Windows:** Use WSL (Windows Subsystem for Linux)

Your First Command

```
echo "Hello, World!"
```

Check Your Shell

```
echo $SHELL
```

Basic Navigation Commands

Where Am I?

```
pwd      # Print Working Directory
```

What's Here?

```
ls        # List files and directories  
ls -l     # Long format (detailed)  
ls -la    # Include hidden files
```

Moving Around

```
cd /path/to/directory # Change directory  
cd ..                 # Go up one level  
cd ~                  # Go to home directory  
cd -                   # Go to previous directory
```

File and Directory Operations

Creating

```
mkdir my_project      # Create directory
touch file.txt         # Create empty file
```

Copying and Moving

```
cp file.txt backup.txt      # Copy file
cp -r folder/ backup_folder/ # Copy directory
mv file.txt new_name.txt    # Move/rename file
```

Removing (Be Careful!)

```
rm file.txt             # Remove file
rm -r folder/           # Remove directory recursively
rm -rf folder/          # Force remove (dangerous!)
```

Viewing and Editing Files

Viewing File Contents

```
cat file.txt          # Display entire file
less file.txt         # View file page by page
head file.txt         # First 10 lines
tail file.txt         # Last 10 lines
tail -f logfile.txt   # Follow file changes (logs)
```

Quick Editing

```
nano file.txt         # Simple text editor
vim file.txt          # Advanced editor
```

Text Processing Power Tools

Searching

```
grep "pattern" file.txt      # Find lines with pattern
grep -r "pattern" folder/    # Search recursively
grep -i "pattern" file.txt    # Case insensitive
```

Sorting and Counting

```
sort file.txt                # Sort lines
sort -r file.txt              # Reverse sort
wc file.txt                   # Word, line, character count
wc -l file.txt                # Line count only
```

Text Manipulation

```
cut -d',' -f1 data.csv       # Extract first column from CSV
sed 's/old/new/g' file.txt    # Replace text
```

Pipes and Redirection

Pipes (|) - Chain Commands

```
ls -la | grep ".txt"           # List only .txt files
cat access.log | grep "ERROR" | wc -l # Count errors
ps aux | grep python          # Find Python processes
```

Redirection

```
echo "Hello" > file.txt        # Write to file (overwrite)
echo "World" >> file.txt        # Append to file
command 2> error.log           # Redirect errors
command > output.txt 2>&1      # Redirect both output and errors
```


Variables and Environment

Creating Variables

```
name="John Doe"           # No spaces around =  
age=25  
echo "Hello, $name"  
echo "You are $age years old"
```

Environment Variables

```
echo $HOME                # Home directory  
echo $PATH                # Executable paths  
echo $USER                # Current user  
export MY_VAR="value"     # Make variable available to subshells
```

Command Substitution

Control Structures - Conditionals

If Statement

```
#!/bin/bash
if [ -f "file.txt" ]; then
    echo "File exists"
elif [ -d "folder" ]; then
    echo "Directory exists"
else
    echo "Neither exists"
fi
```

Common Test Conditions

```
[ -f file ]      # File exists
[ -d directory ] # Directory exists
[ -z "$var" ]    # Variable is empty
[ "$a" = "$b" ]  # Strings are equal
[ "$num" -gt 10 ] # Number greater than 10
```

Control Structures - Loops

For Loop

```
#!/bin/bash
for file in *.txt; do
    echo "Processing $file"
    cp "$file" "backup_$file"
done

for i in {1..5}; do
    echo "Number: $i"
done
```

While Loop

```
#!/bin/bash
counter=1
while [ $counter -le 5 ]; do
    echo "Count: $counter"
```

Functions

Defining Functions

```
#!/bin/bash
greet() {
    local name=$1
    echo "Hello, $name!"
}

backup_file() {
    local file=$1
    if [ -f "$file" ]; then
        cp "$file" "${file}.backup"
        echo "Backed up $file"
    else
        echo "File $file not found"
    fi
}
```

Real-World Example 1: Log Analysis

Analyze Web Server Logs

```
#!/bin/bash
log_file="/var/log/apache2/access.log"

echo "=== Log Analysis Report ==="
echo "Total requests: $(wc -l < $log_file)"
echo "Unique IPs: $(awk '{print $1}' $log_file | sort | uniq | wc -l)"
echo "404 errors: $(grep ' 404 ' $log_file | wc -l)"
echo "Top 5 IPs:"
awk '{print $1}' $log_file | sort | uniq -c | sort -nr | head -5
```

Real-World Example 2: Project Setup

Automated Project Setup Script

```
#!/bin/bash
project_name=$1

if [ -z "$project_name" ]; then
    echo "Usage: $0 <project_name>"
    exit 1
fi

echo "Creating project: $project_name"
mkdir -p "$project_name"/{src,tests,docs,config}
touch "$project_name"/README.md
touch "$project_name"/src/main.py
touch "$project_name"/tests/test_main.py

echo "Project structure created:"
tree "$project_name"
```

Real-World Example 3: System Monitoring

Simple System Health Check

```
#!/bin/bash
echo "=== System Health Check ==="
echo "Date: $(date)"
echo "Uptime: $(uptime)"
echo "Disk Usage:"
df -h | grep -E '^/dev/'
echo "Memory Usage:"
free -h
echo "Top 5 CPU consuming processes:"
ps aux --sort=-%cpu | head -6
```

Best Practices

1. Always Use Shebang

```
#!/bin/bash  
# This tells the system which interpreter to use
```

2. Quote Your Variables

```
# Good  
if [ -f "$filename" ]; then  
    echo "File: $filename"  
fi  
  
# Bad (can break with spaces)  
if [ -f $filename ]; then  
    echo File: $filename  
fi
```


Best Practices (Continued)

4. Use Meaningful Variable Names

```
# Good
user_count=$(wc -l < users.txt)
backup_directory="/backup/$(date +%Y%m%d)"
```

```
# Bad
n=$(wc -l < users.txt)
dir="/backup/$(date +%Y%m%d)"
```

5. Add Comments

```
#!/bin/bash
# Script: backup_project.sh
# Purpose: Create daily backup of project files
# Author: Your Name
# Date: $(date)
```

Useful Commands for Students

File Finding

```
find . -name "*.py"           # Find Python files
find . -type f -size +1M      # Find files larger than 1MB
find . -mtime -7             # Find files modified in last 7 days
```

Process Management

```
ps aux                       # List all processes
kill PID                     # Terminate process
killall process_name        # Kill all instances
jobs                         # List background jobs
bg / fg                     # Background/foreground jobs
```

Compression

```
tar -czf backup.tar.gz folder/ # Create compressed archive
```

Getting Help

Built-in Help

```
man command_name      # Manual page
command_name --help    # Quick help
type command_name      # Show command type
which command_name      # Show command location
```

Online Resources

- **Bash Manual:** gnu.org/software/bash/manual/
- **ShellCheck:** shellcheck.net (script validation)
- **Explain Shell:** explainshell.com
- **Stack Overflow:** stackoverflow.com

Practice Exercises

Exercise 1: File Organization

Create a script that organizes files by extension:

- Move all `.txt` files to `text_files/`
- Move all `.jpg` files to `images/`
- Create a log of moved files

Exercise 2: Backup Script

Write a script that:

- Creates timestamped backups
- Only backs up modified files
- Sends email notification when done

Summary

What We Covered

-  Basic navigation and file operations
-  Text processing and pipes
-  Variables and environment
-  Control structures (if, for, while)
-  Functions and scripting
-  Real-world examples
-  Best practices

Next Steps

- Practice with daily tasks
- Write automation scripts for your projects

Questions?

Contact Information

- **Office Hours:** [Your office hours]
- **Email:** [Your email]
- **Course Website:** [Course URL]

Remember

- **Practice makes perfect**
- **Start small, build complexity**
- **Use version control for your scripts**
- **Always test scripts before using on important data**

Happy scripting! 